

the following command in a terminal:

```
$ espanso start
```

If the installation is successful, you will see a notification that Espanso is running. Now type `:espanso` in any text field, and it will get replaced by the text 'Hi there!'

It is important to know where the configuration file for Espanso is located. For that, enter the following command:

```
$espanso path
```

The above command will show the location of the Espanso files, namely:

- **Config:** This shows the location of the configuration file, which contains the code for how Espanso will behave.
- **Packages:** This location contains the additional packages, which will expand the capabilities of Espanso further.
- **Data:** This shows the location of the other data files necessary for Espanso.

Understanding triggers and matches

Espanso is built on the core components 'Triggers' and 'Matches'. Put in simple terms, in Espanso you can create a trigger and match, so that whenever Espanso meets a trigger the corresponding match is replaced. Let us understand this with the following example.

Imagine you're sending an email to your superior containing a report of your day's work. That email will contain the following parts:

- Greetings to the receiver
- Date of the report
- Message in the body of the email
- Signature of the sender

In the above list, the greetings to the receiver and signature of the sender are almost constant. The rest of the list is somewhat dynamic. Now let's create a trigger and match for greetings to the receiver.

Open the `default.yml` file, which is located in the configuration folder of Espanso. The configuration file offers very few triggers and matches, but you can extend them as per your requirements. Before editing the `default.yml` file, you must understand that indentation is important. It is only through proper indentation that you can segregate the various triggers and matches. So let's place the following lines in the bottom of the `default.yml` file:

```
- trigger: ":greet_will"
  replace: "Hello William Parker"
```

After that, Espanso has to reload the configuration so that new changes may take effect. This is achieved by using a file watcher. Once you save any changes you make in the configuration file, Espanso will automatically reload the configuration file.

Now if you type `greet_will`, it will be replaced by the words *Hello William Parker*.

Let's now create a trigger and match for the signature part of the email. This time you will want to make the words or sentences appear in separate lines:

```
- trigger: ":sig"
  replace: "Nate Alexander,\nDeputy Manager\nQuality Assurance Department"
```

The above code will produce the following output:

```
Nate Alexander,
Deputy Manager
Quality Assurance Department
```

The text inside the `replace` part can be of any length. This offers flexibility to use the chunks of text of variable sizes as per the need.

Till now, you have worked with the static parts of the email. Now let's take up the dynamic parts of the email.

In our email example above, you may wish to add the date of the report you're about to send. The date can be

written as *July 30, 2021*. Instead of that, you can create a trigger named `:present_date` as follows:

```
- trigger: ":present_date"
  replace: "{{mydate}}"
  vars:
    - name: mydate
      type: date
      params:
        format: "%B %d, %Y"
```

Compared to the previous code examples, this one will be lengthy. Let's break it down into parts so that you can understand better.

- 1) **- trigger: "":present_date":** This defines a trigger named `:present_date`.
- 2) **replace: {{mydate}}:** In this line, instead of directly supplying the substitution, the variable is declared to hold the dynamic content. In this case, it is 'mydate'.
- 3) **vars:**

```
- name: mydate
  type: date
```

The above block shows the definition and type of the variable. Type of variable in this context simply means what extension will be executed to produce output for the variable `mydate`. In our example, you are using the `date` extension:

- 4) **params:**

```
format: "%B %d, %Y"
```

The above code shows the parameters for the variable. In this example, you are using the parameter `format`, which will decide how the date shows up. There are many options for formatting the date, where you can customise it according to your needs.

Now if you type `:present_date` in any text area, you will get the following output:

```
July 29, 2021
```